15

LEVEL 15 · COMPLETE PREP GUIDE

GenAI / AI Engineer Interview Prep

Har module, har topic — Roman Urdu mein samjhaya gaya, saath mein "interview mein aise bolein" model answers, debugging frameworks, system-design walkthroughs aur full mock interview loop.

10	60+	40+	3
MODULES	TOPICS EXPLAINED	MODEL ANSWERS	MOCK ROUNDS

SHURU KARNE SE PEHLE

Ye Level Kya Hai aur Is PDF Ko Kaise Use Karein

Level 15 naye concepts nahi sikhata. Ye aap ke pichle 14 levels ke ilm ko **interview performance** mein convert karta hai.

00

Roadmap ka ye aakhri level hai. Is mein focus hai: **scenario-based sawal**, **system design discussions**, **debugging cases**, **trade-offs**, aur apne engineering decisions ko pressure mein **defend** karna. Concept aap ko aata hai — lekin interview mein usay clearly, structured tareeqe se, 3 minute mein bolna ek alag skill hai.

Interview weight: 9/10

Ye level seedha aap ke knowledge ko **answers** mein badalta hai. Trade-offs aur project decisions ko pressure mein explain karna alag skill hai — aur **strong candidates bhi bina practice ke fail ho jaate hain**.

On-the-job weight: 1/10

Job par iska direct use nahi — siwaye is ke ke kabhi aap khud kisi candidate ka interview lein. Yani ye pure **conversion skill** hai.

Is PDF mein har topic ka structure

INTERVIEW MEIN AISE BOLEIN

Ready-made model answer jo aap apne alfaaz mein dohra sakte hain.

PRO TIP

Wo cheez jo aap ko average candidate se alag karti hai.

COMMON GALTI

Wo jawab jo interviewer ko red flag lagta hai.

EXAMPLE

Real scenario jis par concept apply hota hai.

Modules ka naqsha

#	Module	Aap kya seekhenge
M15.1	Interview strategy & role positioning	JD parhna, role samajhna, apna prep plan banana
M15.2	Production RAG interview scenarios	Scale par RAG whiteboard karna aur debug karna
M15.3	Agentic AI interview scenarios	Agent design, failure modes, guardrails, HITL
M15.4	Fine-tuning & model adaptation	Prompt vs RAG vs FT, LoRA/QLoRA trade-offs, deployment
M15.5	LLMOps & production debugging	Incidents, latency, cost, monitoring, rollback

M15.6	AI system-design interviews	Ambiguous brief se full architecture tak
M15.7	Staying current without hype	Naye models ko evaluate karne ka process
M15.8	Project & portfolio defense	Projects ko engineering decisions ki tarah defend karna
M15.9	Behavioral communication	Apni technical thinking ko visible banana
M15.10	Capstone: Full mock interview loop	3 rounds — scenarios, system design, project defense

GOLDEN RULE — POORI PDF KA NICHOR

Har interview answer in 4 qadmon mein do: **(1) Clarify** — sawal ko apne alfaaz mein dohrao aur assumptions batao. **(2) Simplest first** — sab se simple kaam karne wala solution pehle. **(3) Trade-offs** — har choice ke saath "is ka faida ye, nuqsaan ye". **(4) Measure** — kaise pata chalega ke ye kaam kar raha hai (evals, metrics). Jo candidate ye 4 cheezein karta hai, wo "tool names ratne wale" candidate se hamesha aage nikalta hai.

MODULE M15.1 · 1 SESSION

Interview Strategy & Role Positioning

Har cheez tayyar karne ki koshish mat karo. Samjho ke kya test ho raha hai, aur apni existing skills ko us role ke hisaab se position karo.

01

Outcome: Interview preparation ko intelligently prioritize karna aur apni existing skillset ko target role ke liye position karna.

1.1 GenAI interviews kaise badal gaye hain

2023 tak GenAI interviews zyada tar **theory-heavy** the: "Transformer kya hai?", "Attention ka formula likho", "Embeddings kya hoti hain?". Ab ye sawal basic filter ban gaye hain. Aaj ka interviewer janna chahta hai ke **kya aap ne kabhi real system banaya aur chalaya hai?** Is liye sawal ab is tarah ke hote hain:

- **System design:** "10 lakh documents par ek enterprise search assistant design karo."
- **Debugging:** "RAG ek hafta theek chala, phir quality gir gayi. Kya karoge?"
- **Architecture:** "Yahan agent chahiye ya simple workflow?"
- **Production:** "Provider ki latency double ho gayi, users complain kar rahe hain — pehle 10 minute mein kya karoge?"

Wajah simple hai: LLM ab API ke zariye sab ko available hai. Model call karna koi skill nahi. **Skill ye hai ke us call ke ird gird reliable, sasta, tez aur safe system kaise banaya jaye.**

Purana interview (theory)	Naya interview (production)
Self-attention ka formula?	Context window bhar raha hai, quality gir rahi hai — kya karoge?
RAG kya hota hai?	RAG ko 5,000 concurrent users ke liye scale karo
LoRA define karo	Fine-tuned model deploy ke baad kharab kyun chal raha hai?
Agent kya hota hai?	Agent infinite loop mein phans kar $400 jala gaya — fix karo

PREP KA MATLAB

Apna waqt roughly **20% theory revision** aur **80% scenarios, design aur project defense** par lagao. Theory ka jawab 60 second mein aana chahiye; asal waqt scenarios mein lagta hai.

1.2 Job description (JD) parhna

JD mein aksar 20 tools likhe hote hain — LangChain, LlamaIndex, Pinecone, Weaviate, AWS, Azure, Kubernetes, Docker... Naya candidate sochta hai ke sab seekhna parega. **Ghalat.** JD ko decode karo aur dekho ke roadmap ke kaun se **areas** asal mein test ho rahe hain.

3-bucket method

Must-have

Jo cheezein JD ke pehle hisse mein hain, bar bar repeat hoti hain, ya "required" mein likhi hain. Yahan 70% waqt lagao.

Nice-to-have

"Preferred", "bonus", "plus". Inka concept clear rakho, deep dive zaroori nahi.

Noise / buzzwords

Tools ki lambi list. Tool ko **concept** mein translate karo — Pinecone = "vector database".

JD lines ko roadmap levels se map karna

JD mein likha hai	Asal mein kya test hoga	Roadmap level
"Build RAG pipelines over enterprise documents"	Ingestion, chunking, retrieval quality, scale	L4, L9
"Experience with LangGraph / agent frameworks"	Agent design, tools, failure handling	L5, L6
"Evaluate and improve LLM output quality"	Eval sets, LLM-as-judge, regression testing	L7
"Deploy on AWS/Azure/GCP"	LLMOps, CI/CD, observability, managed platforms	L10, L11
"Ensure safety and compliance"	Prompt injection, PII, guardrails, IAM	L8
"Fine-tune open-source models"	LoRA/QLoRA, data prep, serving	L13

INTERVIEW MEIN AISE BOLEIN

"Main ne JD dekhi — mujhe laga ke is role ka core **production RAG aur evaluation** hai, kyunke ye dono pehle teen bullets mein hain. Main ne Pinecone use nahi kiya, lekin pgvector aur FAISS par kaam kiya hai — concept wahi hai: ANN index, metadata filtering, aur recall-vs-latency trade-off."

COMMON GALTI

"Mujhe ye tool nahi aata" kehkar ruk jana. Hamesha tool ko concept se replace karo aur batao ke aap ne wahi concept kahan use kiya.

1.3 AI Engineer vs GenAI Data Scientist vs Data Scientist

Ye teen bilkul alag disciplines nahi — farq **emphasis** ka hai. Teeno ko data, models aur evaluation aana chahiye; lekin har role ka "center of gravity" alag hai.

Pehlu	AI Engineer	GenAI Data Scientist	Traditional DS
Main focus	LLM-powered products banana aur production mein chalana	LLM ki quality, evaluation, experiments, fine-tuning	Classical ML, statistics, business analytics

Rozana kaam	APIs, RAG, agents, deployment, monitoring, cost	Eval design, data curation, prompt/FT experiments	Feature engineering, forecasting, A/B tests, dashboards
Interview emphasis	System design, debugging, LLMOps, reliability	Evals, metrics, FT trade-offs, experiment design	Stats, SQL, ML algorithms, case studies
Code style	Production backend (Python/TS), services	Notebooks + pipelines	Notebooks, SQL

POSITIONING

Apne aap ko us role ke center par rakho jis ke liye apply kar rahe ho. Agar AI Engineer role hai, to apne projects ka **deployment, latency, cost aur reliability** wala hissa highlight karo — sirf "model ki accuracy" nahi.

1.4 Engineering layer kyun dominate karti hai

Aaj har company ke paas wahi frontier models hain. Is liye **competitive advantage model mein nahi, system mein hai**. Interviews isi liye in cheezon par zor dete hain:

APIs & inference

Rate limits, streaming, batching, token budgets.

LLMOps

Prompt/model versioning, eval gates, CI/CD.

Observability

Traces, latency p95, cost per request, quality drift.

Networking & cloud

VPC, private endpoints, regions, IAM.

Reliability

Retries, fallbacks, circuit breakers, SLAs.

Deployment

Containers, autoscaling, canary, rollback.

INTERVIEW MEIN AISE BOLEIN

"Demo aur product mein farq engineering ka hai. Demo mein ek LLM call 90% theek chalti hai. Production mein mujhe us 10% ka plan chahiye — timeouts, rate limits, ghalat retrieval, provider outage — aur ye bhi pata hona chahiye ke har request ki cost aur latency kya hai."

1.5 API-hosted vs self-hosted (open-weight) models

Ye sawal almost har interview mein aata hai. Jawab "depends" hai — lekin **kis cheez par depend** karta hai, ye clearly batana hai. Chhe dimensions par compare karo:

Dimension	API-hosted (OpenAI, Anthropic, Gemini)	Self-hosted (Llama, Qwen, Mistral on own GPUs)
Capability	Usually best frontier quality, reasoning, tools	Strong lekin aksar thoda peeche; narrow task par FT se barabar ho sakta hai
Cost	Pay-per-token; kam/uneven traffic par sasta	Fixed GPU cost; high, steady volume par per-token sasta

Latency	Network + provider queue; aap control nahi kar sakte	Apne hardware par tune kar sakte ho; same-region, no internet hop
Control	Model kabhi bhi update/deprecate ho sakta hai	Version freeze, custom fine-tuning, full control
Privacy	Data third party ke paas jata hai (agreements ke saath)	Data aap ke network se bahar nahi jata
Ops burden	Bohat kam — bas API call	Zyada — GPUs, serving (vLLM/TGI), scaling, monitoring, on-call

INTERVIEW MEIN AISE BOLEIN

"Main default mein **API-hosted** se shuru karunga kyunke time-to-market fast hai aur ops burden kam. Self-hosting par tab jaunga jab in mein se koi condition sach ho: (1) traffic itna high aur steady ho ke GPU cost per-token API se kam ho jaye, (2) data residency ya privacy ki wajah se data bahar nahi ja sakta, (3) mujhe fine-tuned ya frozen model chahiye jis ka version provider na badle. Aur ye decision main reversible rakhunga — ek model gateway ke peeche, taake baad mein switch karna asaan ho."

BONUS POINT

"Middle option" zaroor mention karo: **managed platforms** (Azure OpenAI, AWS Bedrock, Vertex AI) — frontier models aap ke cloud tenant aur region mein, private networking ke saath. Bohat si enterprises yehi choose karti hain.

1.6 Apna personalized preparation plan

Pehle target role decide karo, phir apni weaknesses honestly rate karo (1 = bilkul nahi aata, 5 = confidently whiteboard kar sakta hoon). Sab se kam score wale, lekin JD mein important areas, pehle.

Area	Self-score (1–5)	JD mein importance	Priority
Production RAG design & debugging	___	High / Med / Low	___
Agent design & failure modes	___	___	___
Evals & monitoring	___	___	___
LLMOps, cloud, deployment	___	___	___
Fine-tuning & serving	___	___	___
System design communication	___	___	___
Project defense (2–3 projects)	___	___	___

4-hafte ka sample plan (AI Engineer role ke liye)

Hafta	Focus	Deliverable
Week 1	M15.1 + M15.2 (RAG) + M15.8 (projects likhna)	2–3 projects ka 9-step write-up; 1 RAG design whiteboard
Week 2	M15.3 (agents) + M15.5 (LLMOps debugging)	10 debugging scenarios ke bolay hue answers (record karo)
Week 3	M15.6 (system design) + M15.4 (fine-tuning)	4 practice systems design, har ek 35 minute mein
Week 4	M15.7 + M15.9 + M15.10 (mock loops)	Kam az kam 3 full mock interviews, feedback ke saath

COMMON GALTI

Sirf parhte rehna. Interview prep ka 50% hissa **bol kar practice** hona chahiye — phone par record karo, sun kar dekho ke kahan atke, kahan jargon zyada tha.

MODULE M15.2 · 2 SESSIONS

Production RAG Interview Scenarios

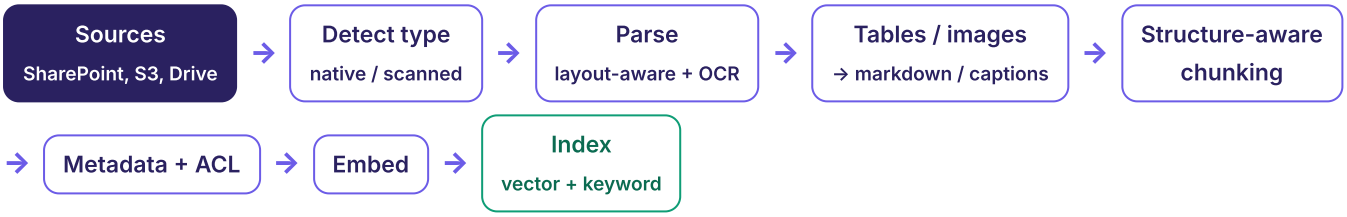
"Retriever → LLM" wala demo sab bana lete hain. Interview mein aap ko ek production-scale RAG whiteboard aur defend karna hai.

02

Outcome: Production-scale RAG architecture whiteboard aur defend karna — demo-level retriever → LLM pipeline describe karne ke bajaye.

2.1 Messy enterprise data ke liye ingestion

Asal duniya ka data saaf text nahi hota. Enterprise mein complex **PDFs** hote hain (scanned bhi), nested tables, diagrams, har document ka layout alag, aur documents roz badalte rehte hain. **RAG ki 60–70% quality problems ingestion se aati hain** — retrieval ya LLM se nahi.



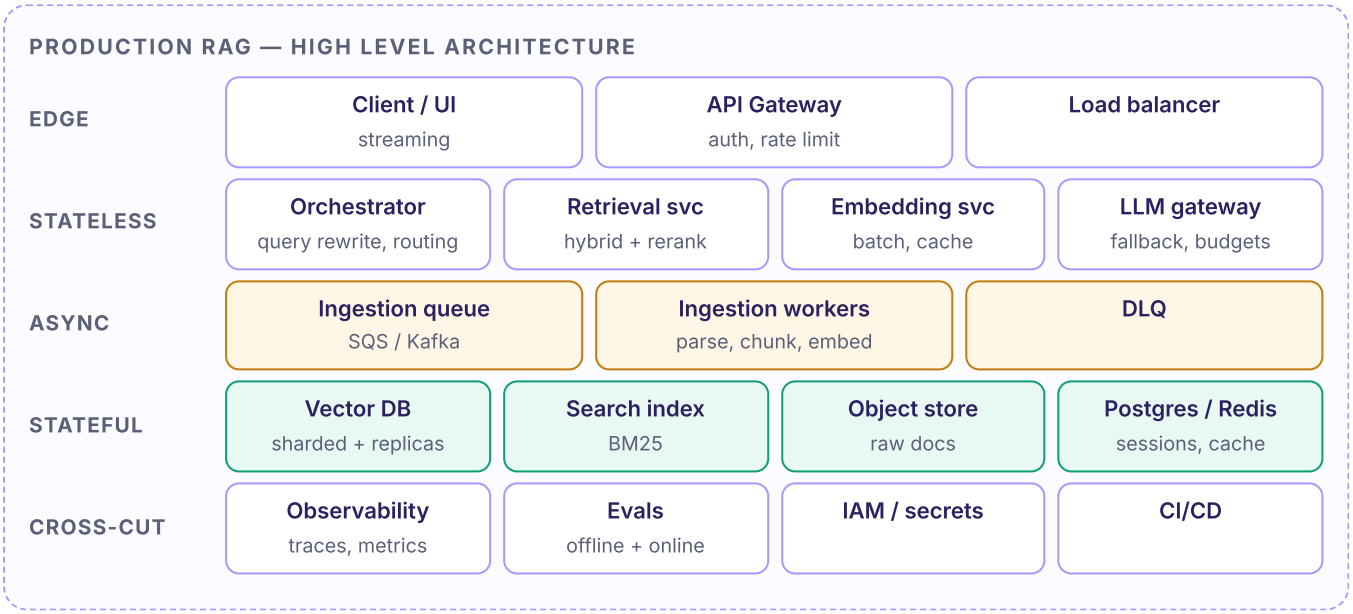
Problem	Solution
Scanned PDFs	OCR (e.g., Tesseract, cloud document AI). Pehle detect karo ke page mein text layer hai ya nahi.
Nested tables	Layout-aware parser se table ko markdown/HTML mein convert karo; table ko beech se mat todo; header row har chunk mein repeat karo.
Diagrams / charts	Vision-language model se caption ya description generate karo, aur usay text ki tarah index karo (image link metadata mein).
Varying layouts	Headings/sections ke hisaab se chunking (structure-aware), fixed 500-token cutting nahi.
Documents change hote hain	Content hash se change detect karo; sirf badle hue chunks re-embed; purane delete; `doc_id + version` track karo.
Duplicates	Hash / near-duplicate detection — warna retrieval mein ek hi cheez 5 baar aayegi.
Parse failures	Dead-letter queue (DLQ) mein daalo, alert do, retry karo — poori pipeline mat roko.

INTERVIEW MEIN AISE BOLEIN

"Main ingestion ko ek **async, idempotent pipeline** banaunga. Har document pehle type-detect hoga, phir layout-aware parser chalega; tables markdown mein, diagrams VLM caption ke saath. Har chunk ke saath metadata jayega — source, page, section, version, aur **access-control list** — taake retrieval time par user sirf wahi dekhe jis ki use permission hai. Updates ke liye content hash se sirf changed chunks re-embed honge."

2.2 Concurrent users ke scale par RAG

Interviewer kehta hai: "50,000 employees, 2 crore chunks, peak par 2,000 requests per minute." Aap ko batana hai ke system kahan tootega aur kaise scale karega.



Scale ke key levers

- **LLM aam tor par bottleneck hota hai** (latency aur cost dono). Streaming responses, prompt ko chhota rakhna, semantic cache, aur multiple providers/regions.
- **Vector DB:** ANN index (HNSW/IVF), sharding (data bara ho to), read replicas (QPS zyada ho to), metadata pre-filtering.
- **Embedding service:** query embeddings ka cache; batching; GPU ya API.
- **Caching:** exact-match cache, semantic cache (similar sawal), retrieval-result cache — lekin ACL aur freshness ka khayal.
- **Rate limiting & backpressure:** per-user/tenant limits taake ek user poora system na bhar de.

NUMBERS BOLNA SEEKHO

"2 crore chunks × 1024-dim × 4 bytes ≈ 80 GB sirf vectors ke liye — ek machine ki RAM par fit nahi, is liye sharding ya quantized vectors (PQ) chahiye." Ek rough calculation aap ko senior dikhati hai.

2.3 Architecture ko services mein todna

Service	Kaam	Kyun alag?
Ingestion	Parse, chunk, metadata, index updates	Batch/async workload; spikes; alag scaling
Embedding	Text → vectors (ingest aur query dono)	GPU ya API; independent scaling aur model versioning
Retrieval	Hybrid search, filters, reranking	Latency-critical; CPU/memory heavy
LLM inference	Answer generation (API ya self-hosted)	Sab se mehnga aur slow; fallbacks yahan
Orchestration	Query rewrite, routing, prompt assembly, guardrails	Business logic; jaldi jaldi change hota hai
Storage	Vector DB, object store, metadata DB, cache	State yahan rehta hai; backups, durability

Alag services ka faida: **independent scaling** (GPU vs CPU), **independent deploys** (prompt change ke liye vector DB redeploy nahi), aur **failure isolation** (reranker gira to basic retrieval phir bhi chale).

COMMON GALTI

Pehle hi din 12 microservices bana dena. Interview mein bolo: "MVP mein main ek modular monolith rakhunga, lekin boundaries in services ke hisaab se — jab scaling ki zaroorat aaye, alag kar dunga."

2.4 Stateless services, stateful stores

Stateless ka matlab: service apni memory mein koi user-specific cheez yaad nahi rakhti. Har request apne saath zaroori context laati hai, ya service usay database se uthati hai. Faida: koi bhi replica koi bhi request serve kar sakta hai, to **horizontal scaling, restart, aur autoscaling** asaan.

State ki qisam	Kahan rakhein
Raw documents, files	Object store (S3, Blob, GCS)
Embeddings	Vector database (pgvector, Qdrant, Pinecone, Weaviate)
Keyword index	Search index (Elasticsearch/OpenSearch)
Entities / relationships	Graph database (Neo4j) — GraphRAG ke liye
Chat history, user sessions	Postgres ya Redis
Hot cache, rate-limit counters	Redis
Job status (ingestion)	DB + queue

INTERVIEW MEIN AISE BOLEIN

"Orchestrator aur retrieval services stateless rahengi taake main unhein Kubernetes par pod count badha kar scale kar sakoon. Conversation history Redis/Postgres mein jayegi, session id ke saath. Agar koi pod crash ho, user ko farq nahi parega — agli request kisi aur pod par chali jayegi."

2.5 RAG ke ird gird production infrastructure

Ye 7 cheezein interviewer sunna chahta hai. Har ek ka ek line ka "kyun" yaad rakho:

1. Asynchronous messaging & queues

Ingestion aur lambe kaam queue (SQS, Kafka, RabbitMQ, Celery) se. Faida: spikes absorb, retries, backpressure, aur API fast rehti hai. Failed jobs DLQ mein.

2. Service fallbacks

Primary LLM down → secondary provider/model. Vector DB slow → BM25 keyword search. Reranker fail → bina rerank top-k. Sab fail → honest "abhi jawab nahi de sakta" + cached FAQ.

3. Object / vector / graph / search storage

Har data ki apni jagah (upar wala table). Hybrid search (vector + BM25) exact codes, names aur IDs ke liye zaroori.

4. Observability & SLAs

Har request ka trace: retrieval latency, retrieved chunks, prompt tokens, LLM latency, cost. SLO example: p95 < 4s, 99.9% availability, faithfulness > 0.9.

5. Networking & IAM

Services private VPC mein, LLM providers tak private endpoints. Least-privilege IAM roles. **Document-level ACL** retrieval mein filter ki tarah lagao — warna HR ka data intern ko mil jayega.

6. Secrets & encryption

API keys secret manager mein (code ya env file mein nahi). TLS in transit, KMS se encryption at rest. Keys rotate karo.

7. CI/CD & deployment

Prompts aur configs version control mein. Har change par **eval gate** chale (golden set par faithfulness/recall girna nahi chahiye). Canary ya blue-green deploy. Index bhi versioned: naya embedding model aaye to naya index alag banao, phir traffic switch karo (dono indexes mix mat karo).

2.6 Debugging: "RAG ek hafta theek chala, phir quality gir gayi"

Ye **classic** sawal hai. Interviewer dekhna chahta hai ke aap random cheezein try karte ho ya **structured investigation** karte ho.

1. **Clarify:** "Quality gir gayi" kaise pata chala — user complaints, thumbs-down rate, ya eval scores? Achanak gira ya dheere dheere? Sab queries mein ya kisi khaas topic/department mein?
2. **Kya badla? (change log):** Code/prompt deploy? Model version (provider ne silently update kiya)? Embedding model? Naye documents aaye? Index rebuild hua? Config (top-k, chunk size)?
3. **Data check:** Ingestion failures (DLQ bhar gayi?), naye docs parse nahi hue, duplicates, purane versions delete nahi hue, metadata/ACL ghalat.

4. **Query distribution shift:** Users ab naye qisam ke sawal pooch rahe hain jo golden set mein nahi (e.g., naye product launch ke baad).
5. **Stage isolate karo:** neeche table — retrieval ya generation?
6. **Fix + guard:** root cause fix, aur us case ko eval set mein add karo taake dobara na ho; monitoring alert lagao.

2.7 Failing stage ko isolate karna

Stage	Symptom	Kaise check karein
Ingestion	Naye docs ke sawalon ka jawab nahi milta	Index mein doc count vs source count; parse error logs; DLQ
Data change	Purani/contradicting info	Kya old versions delete hue? Same topic ke 2 versions retrieve ho rahe?
Retrieval	Ghalat ya irrelevant chunks	Golden set par recall@k / MRR; failing queries ke retrieved chunks manually dekho
Generation	Sahi chunks mile, phir bhi jawab ghalat / hallucinated	Sahi context manually de kar LLM test karo; faithfulness score
Model change	Style/format achanak badla	Model version pin hai? Provider changelog
Infrastructure	Jawab adhoore, timeouts	Context truncation, token limits, reranker timeout ke baad fallback chal raha?

INTERVIEW MEIN AISE BOLEIN

"Main sab se pehle problem ko **retrieval vs generation** mein divide karunga. Failing queries ke traces nikalunga: agar sahi chunk top-k mein hai hi nahi, to masla retrieval ya ingestion ka hai; agar sahi chunk tha aur jawab phir bhi ghalat, to generation/prompt/model ka. Saath hi pichle hafte ka change-log dekhunga — 80% time koi deploy, data update ya model version change hi wajah hoti hai."

COMMON GALTI

Seedha "chunk size badal dunga" ya "behtar model laga dunga" kehna. Bina diagnose kiye fix batana sab se bara red flag hai.

MODULE M15.3 · 2 SESSIONS

Agentic AI Interview Scenarios

Agent design karna aur defend karna — autonomy, cost, reliability, recovery aur "blast radius" par explicitly soch kar.

03

Outcome: Agent architecture design aur defend karna, autonomy, cost, reliability, recovery aur blast radius ke baare mein explicitly reason karte hue.

3.1 Business task ke liye agent design karna

Interviewer ek concrete task deta hai, jaise: *"E-commerce company ke liye customer support agent banao jo order status bataye, refunds process kare aur complaints escalate kare."* Har agent ke 6 building blocks hote hain:

Goal & scope

Kya karna hai, aur kya **nahi** karna. Success kaise measure hoga.

Planning

Task ko steps mein todna — LLM khud plan kare ya fixed workflow?

Tools

`get_order`, `issue_refund`, `create_ticket` — clear schemas ke saath.

Memory

Short-term (is conversation ka state), long-term (customer history, preferences).

Execution loop

Think → act (tool call) → observe → repeat, step aur budget limits ke saath.

Guardrails & HITL

Refund > Rs 5,000 par insaan ki approval; policy checks.



INTERVIEW MEIN AISE BOLEIN

"Pehle main scope fix karunga: agent order status aur Rs 5,000 tak ke refunds khud kar sakta hai; usse upar ya fraud signal ho to human approval. Tools sirf teen honge, har ek ka strict JSON schema. Read-only tools freely call ho sakte hain, write tools idempotent honge. Har run par max 8 steps aur token budget hoga, aur poora trajectory trace hoga taake debugging ho sake."

3.2 Workflow ya autonomous agent?

Workflow mein steps aap code mein pehle se define karte ho (LLM kuch steps mein help karta hai). **Agent** mein LLM khud decide karta hai ke agla step kya ho. Interview mein ye trade-off bohat pooch jata hai.

Pehlu	Deterministic workflow	Autonomous agent
Kab use karein	Steps pehle se maloom; process stable	Open-ended task; steps pehle se pata nahi
Reliability	High, predictable	Kam; har run alag path le sakta hai
Cost / latency	Kam, fixed	Zyada, variable (kai LLM calls)
Debugging	Asaan	Mushkil — trajectory analysis chahiye
Flexibility	Kam	Zyada
Example	Invoice extract → validate → ERP mein daalo	Research agent jo web search karke report likhe

INTERVIEW MEIN AISE BOLEIN

"Mera default **workflow** hai. Agar main flowchart bana sakta hoon, to woh code hona chahiye, LLM ka decision nahi. Agent sirf wahan jahan path pehle se predict nahi ho sakta. Aksar best design hybrid hota hai: bahar ek deterministic workflow, aur sirf ek step ke andar chhota agent."

3.3 Tools, MCP, memory aur multi-agent

Tools

Tools agent ke "haath" hain. Achhe tools: **kam** (10–15 se zyada ho to selection ghalat hone lagta hai), **clear names aur descriptions**, strict input schema, aur **helpful error messages** jo agent ko batayein ke agla kya kare.

MCP (Model Context Protocol)

MCP ek open standard hai jis se tools, resources aur prompts ko ek common tareeqe se expose kiya jata hai — jaise "USB-C for AI tools". Ek baar MCP server bana do (e.g., aap ka Shopify/CRM), to koi bhi MCP-compatible client usay use kar sakta hai. Interview trade-offs: standardization aur reuse vs. extra layer, security (server ko kitni permissions?), aur trust (third-party MCP servers ka tool description bhi prompt injection ka zariya ban sakta hai).

Memory

Type	Kya	Kahan
Working / short-term	Current task ka state, recent messages	Context window + session store
Long-term (semantic)	User facts, preferences, past outcomes	Vector DB / key-value store
Episodic	Pichle runs ke trajectories aur lessons	DB, retrieval se wapas

Multi-agent

Multi-agent tab faida deta hai jab: (1) kaam ko **parallel** kiya ja sake (5 sources ko alag alag research), (2) har sub-task ko **alag, saaf context** chahiye, (3) specialized prompts/tools alag hon. Nuqsaan: tokens kai guna, coordination bugs, debugging mushkil.

COMMON GALTI

"Main 5 agents banaunga — planner, researcher, writer, critic, manager" — bina wajah. Interviewer poochega "single agent kyun nahi?" Jawab tayyar rakho.

3.4 Common failure scenarios (aur un ke fixes)

Interview mein ye saat failures zaroor aate hain. Har ek ka **cause + fix** yaad rakho:

Failure	Kyun hota hai	Fix
Runaway loops	Agent ko lagta hai kaam abhi poora nahi; same tool bar bar; error ko samajh nahi pata	Max-steps limit, token/$ budget, repeated-action detection, "stuck" hone par escalate ya graceful stop
Poor tool selection	Tool descriptions overlap, tools zyada, ghalat naam	Tool descriptions behtar, tools kam karo, routing layer, few-shot examples, tool-selection eval
Tool failures	API down, timeout, ghalat arguments	Retries with backoff (sirf retryable errors), idempotency keys, structured error message agent ko wapas, fallback tool
Nondeterministic trajectories	LLM sampling; same input par alag path	Temperature kam, deterministic hisse code mein, trajectory evals, multiple runs par success rate (pass@k) measure
Excessive token / cost	Lamba context har step par dobara bheja, bara model har kaam ke liye	Per-run budget, context compaction, chhota model sub-tasks ke liye, prompt caching, early stopping
Context degradation	Context lamba hone par model purani instructions bhool jata hai ("lost in the middle")	Summarize/compact purane steps, sub-agents ko saaf context, sirf relevant retrieval, key instructions repeat
Overly powerful tool permissions	Agent ke paas delete/pay/send-all jaisi powers; prompt injection ka khatra	Least privilege, scoped tokens, read-only default, sandbox, dangerous actions par approval, allow-lists

"BLAST RADIUS" LAFZ USE KARO

"Agar ye agent ghalat ho jaye to zyada se zyada kitna nuqsaan ho sakta hai?" Ye sawal khud se poochna aur interviewer ko batana senior thinking dikhata hai. Design ka maqsad: blast radius chhota rakhna.

3.5 Human-in-the-loop checkpoints aur guardrails

Har action ko risk ke hisaab se tier karo:

Risk tier	Examples	Policy
Low — read only	Order status dekhna, docs search	Automatic
Medium — reversible write	Draft email banana, ticket create	Automatic + log + undo option

High — irreversible / money / external	Refund, payment, email bhejna, data delete	Human approval zaroori (ya threshold ke upar)

Guardrails: input checks (prompt injection, PII), output checks (policy violations, format validation), action checks (amount limits, allow-listed recipients), aur audit log.

3.6 Trajectory se agent debug karna

Scenario: *"Agent testing mein 95% kaamyab tha, production mein 60%."* Trajectory = agent ke har step ka record (thought, tool call, arguments, result).

1. Failing runs ke **traces** nikalo (LangSmith, Langfuse, Arize, ya apna logging).
2. Har trace mein dekho: **kis step par** agent sahi path se hata? Ghalat tool? Ghalat argument? Tool ka error samajh nahi aaya?
3. Failures ko **categorize** karo — 50 traces mein se 30 ek hi wajah se hon to wahi pehle fix.
4. **Test vs prod ka farq** dhundo: real inputs messy hain (Roman Urdu, typos), tools slow hain/timeouts, data bara hai, permissions alag, rate limits.
5. Failing cases ko **eval set** mein daalo, fix karo, **replay** karke confirm karo.

INTERVIEW MEIN AISE BOLEIN

"Testing set shayad saaf aur chhota tha. Main production ke 50 failing trajectories dekhunga aur categorize karunga. Mera guess hoga ke ya to real inputs zyada messy hain, ya tools production mein slow hain aur agent timeout ko samajh nahi pa raha. Phir main un real cases ko apne eval set mein shamil karunga taake testing production jaisi ho."

3.7 Kab autonomous/multi-agent NAHI banana

- Jab **ek LLM call + retrieval** se kaam ho jaye.
- Jab process **deterministic** ho (har baar wahi steps).
- Jab **error ki gunjaish zero** ho (finance, medical decisions) aur auditability chahiye.
- Jab **latency budget** tight ho (e.g., 1 second mein jawab).
- Jab cost per request bohat kam rakhni ho aur volume high ho.

INTERVIEW MEIN AISE BOLEIN

"Autonomy ek cost hai, feature nahi. Main use tab add karta hoon jab simple workflow us task ko handle nahi kar sakta — aur tab bhi narrow scope, limited tools aur budget ke saath."

MODULE M15.4 · 1 SESSION

Fine-Tuning & Model Adaptation Interviews

Fine-tuning kab karna hai, kab nahi — aur deploy hone ke baad performance gire to systematically kaise debug karna hai.

04

Outcome: Model-adaptation strategy defend karna aur deployment ke baad performance degradation ko systematically debug karna.

4.1 Prompt vs RAG vs Fine-tuning — product brief se decision

Sab se useful mental model: **RAG knowledge problem solve karta hai, fine-tuning behavior problem.** Aur dono se pehle prompt engineering.

Masla kya hai?	Pehla solution	Kyun
Model ko company ka data / latest info nahi pata	RAG	Knowledge bahar rehti hai, update asaan, citations milti hain
Output format / tone inconsistent hai	Prompt + few-shot → phir FT	Pehle sasta tareeqa; consistency phir bhi na aaye to FT
Narrow task, high volume, cost bohat zyada	Fine-tune chhota model (distillation)	Bara model ka kaam chhota model sasta aur tez kare
Specialized language / domain jargon	RAG + FT dono	FT style/jargon, RAG facts
Data jaldi jaldi badalta hai	RAG (FT nahi)	Har update par retrain mumkin nahi
On-prem / offline zaroori	Open-weight model (+ FT agar zaroorat)	Data bahar nahi ja sakta

EXAMPLE BRIEF

Brief: "Legal firm chahti hai ke model contracts ki clauses ko unke internal 40 categories mein classify kare, roz 2 lakh clauses."

Jawab: Pehle frontier model + few-shot prompt se baseline. Agar accuracy theek hai lekin cost/latency zyada (2 lakh/day), to us model ke outputs (human review ke saath) se training data bana kar **chhota model fine-tune** — classification narrow aur stable task hai, FT ke liye perfect. Knowledge ka masla nahi, is liye RAG ki zaroorat nahi.

4.2 Fine-tuning kab apni complexity "kamata" hai

FT ki value (kab worth it)

- Narrow, repeatable task at high volume
- Strict output format jo prompt se consistent nahi
- Latency/cost kam karna (chhota model)
- Style/tone/domain language
- Prompt bohat lamba ho gaya (FT instructions ko "andar" daal deta hai)

FT ki hidden costs

- Quality training data banana (sab se mehnga)
- Training infra aur experiments
- Eval suite aur regression testing
- Serving infra (GPUs) aur monitoring
- Base model update ho to dobara FT
- Naye facts seekhne mein kamzor; hallucination kam nahi karta guarantee se

COMMON GALTI

"Model ko hamara data sikhane ke liye fine-tune karenge." FT facts reliably yaad karwane ka achha tareeqa nahi — wo RAG ka kaam hai. Ye galti interviewer foran pakad leta hai.

4.3 LoRA, QLoRA, PEFT & quantization — trade-offs ki zubaan mein

Definitions sab ko aati hain. Interview mein **trade-off** batao:

Technique	Kya karta hai (1 line)	Faida	Nuqsaan / trade-off
Full fine-tuning	Saare weights update	Max flexibility/quality	Bohat GPU memory, mehnga, har task ke liye poori model copy
PEFT (umbrella)	Sirf thore parameters train	Sasta, tez, kam memory	Bohat bari behavior changes mein kam capacity
LoRA	Frozen weights ke saath chhote low-rank adapter matrices	Adapter sirf kuch MB; ek base model par kai adapters serve	Rank (r) aur target layers tune karne parte hain
QLoRA	Base model 4-bit quantized + LoRA	Bara model chhoti GPU par train (e.g., single 24GB card)	Training thori slow; quantization se halka quality loss mumkin
Quantization (inference)	Weights ko 8/4-bit mein (AWQ, GPTQ, GGUF)	Kam memory, sasti serving, aksar tez	Accuracy drop mumkin — khaas taur par reasoning/math par; eval zaroori

INTERVIEW MEIN AISE BOLEIN

"Main LoRA prefer karunga kyunke ek hi base model GPU par load reh sakta hai aur alag customers ke alag adapters hot-swap ho sakte hain — serving cost bohat kam. Agar GPU budget tight ho to QLoRA se training, lekin deploy se pehle main quantized aur non-quantized versions ko apne eval set par compare karunga, kyunke 4-bit kabhi kabhi edge cases kharab kar deta hai."

4.4 Fine-tuned open-weight model deploy karna

Pehlu	Kya sochna hai
Serving architecture	vLLM / TGI / SGLang jaisa inference server; continuous batching; LoRA adapters multi-tenant serving; model gateway ke peeche
GPU requirements	Rough rule: weights memory ≈ params × bytes. 8B model FP16 ≈ 16 GB, INT4 ≈ 4–5 GB — **plus KV cache** jo context length aur concurrent users ke saath barhta hai
Inference optimization	Quantization, continuous batching, paged attention (KV cache), prefix caching, speculative decoding
Latency	TTFT (time to first token) aur tokens/second per user; streaming
Cost	GPU $/hour ÷ tokens/hour = cost per 1M tokens; utilization kam ho to API sasti parti hai
Scaling	Queue depth / GPU utilization par autoscale; cold start (model load) minutes le sakta hai — warm pool rakho

EK LINE JO IMPRESS KARTI HAI

"Self-hosting ki economics utilization par chalti hai. Agar GPU 20% time busy hai, to per-token cost API se zyada hogi. Main pehle traffic pattern dekhunga."

4.5 Debugging: "Deploy se pehle achha tha, production mein accuracy gir gayi"

Ye bhi classic hai. Investigation ki checklist — **sab se common wajah pehle**:

Kya check karein	Kya ho sakta hai
Prompt / chat template	Training mein ek template (system/user tags) tha, serving mein alag. **Sab se common bug.**
Formatting	Training data mein inputs ka format production inputs se alag (whitespace, JSON vs text, language)
Inference config	Temperature, top-p, max tokens, stop sequences eval aur prod mein alag
Serving changes	Adapter load hi nahi hua / purana version; tokenizer mismatch; context truncation
Quantization artifacts	Eval FP16 par hui, deploy INT4 par — edge cases kharab
Distribution shift	Production users training data se alag sawal pooch rahe hain
Evaluation regression	Eval set training data se leak tha (overfit), ya bohat chhota/easy tha — asal accuracy kabhi thi hi nahi
Training data	Labels noisy, ek class over-represented, ghalat examples

INTERVIEW MEIN AISE BOLEIN

"Sab se pehle main **same eval set ko production serving stack par** chalaunga. Agar wahan bhi score gira hai, to masla serving mein hai — template, config, adapter version ya quantization. Agar eval set par score theek hai, to masla data mein hai — distribution shift ya eval set production ko represent hi nahi karta. Ye ek test problem ko aadha kar deta hai."

MODULE M15.5 · 2 SESSIONS

LLMOps & Production Debugging Interviews

Ye dikhana ke aap AI system ko notebook se bahar nikalne ke baad **chala** aur **theek** bhi kar sakte ho.

05

Outcome: Ye demonstrate karna ke aap AI system ko notebook se nikalne ke baad operate aur troubleshoot kar sakte ho.

5.1 Production incidents diagnose karna

AI system kai cheezon par depend karta hai: model APIs, self-hosted inference, retrieval services, queues, caches, rate limits, aur downstream tools (CRM, payment API). Incident mein in mein se koi bhi wajah ho sakti hai. Har incident ke liye ek hi framework:

1. **Impact:** Kitne users? Kaun se features? Kab se? (Severity decide karo.)
2. **Mitigate pehle, root cause baad mein:** rollback, fallback on, feature flag off — users ka nuqsaan roko.
3. **Timeline + "kya badla":** deploys, config, provider status page, traffic spike, data update.
4. **Layer by layer:** gateway → orchestrator → retrieval → LLM → downstream tools. Traces mein dekho latency/error kahan se aa rahi hai.
5. **Root cause fix + postmortem:** blameless postmortem, alert/eval add karo taake dobara jaldi pakra jaye.

SENIOR SIGNAL

"Pehle mitigate, phir investigate" kehna bohat strong signal hai. Junior candidates seedha root cause dhundne lag jaate hain jab ke users 40 minute se pareshan hain.

5.2 Latency spikes, outages, cost explosions, capacity, quality

Incident	Mumkin wajahein	Immediate action
Latency spike	Provider slow, lambe prompts (context barh gaya), retrieval/reranker slow, queue backlog, cold starts	Traces se slow span dhundo; timeouts + fallback model; context trim; scale up
Provider outage	API 5xx / timeouts	Circuit breaker → secondary provider/region; degraded mode; status page communicate

Cost explosion	Prompt change ne tokens barhaye, agent loops, retry storm, abuse/bot traffic, cache band ho gaya	Per-feature cost dashboards; budgets/quotas; loop limits; rate limit abusers
Capacity limits	Provider rate limit (429), GPU full, TPM quota	Queue + backoff; load spread across keys/regions/providers; quota increase; priority tiers
Degraded quality	Model version change, data drift, prompt bug, retrieval issue	Online eval scores, user feedback; pin model versions; rollback prompt

5.3 Fallbacks, retries, load balancing, autoscaling

Retries

Sirf **retryable** errors par (429, 5xx, timeouts) — 400 par nahi. **Exponential backoff + jitter**, max attempts. Write actions ke liye idempotency keys, warna double refund ho jayega.

Circuit breaker

Agar provider lagataar fail ho raha hai to thori der ke liye usay call karna band karo aur seedha fallback par jao — retry storm se bacho.

Fallbacks & graceful degradation

Primary model → secondary model → cached answer → honest message. Feature-level: reranker off, sirf top-3 chunks, chhota model. System "gire" nahi, bas thora kam achha ho.

Load balancing & autoscaling

Multiple API keys/regions/providers par traffic. Self-hosted GPUs ke liye autoscale **queue depth ya GPU utilization** par, CPU par nahi. Warm pool rakho kyunke model load hone mein minutes lagte hain.

5.4 Monitoring signals parhna

Signal	Kya batata hai	Example alert
Latency (p50 / p95 / p99, TTFT)	User experience; tail latency asal dard hai	p95 > 6s for 10 min
Throughput (RPS, tokens/s)	Load aur capacity	Traffic 3× normal
Error rate (4xx, 5xx, 429, timeouts)	Reliability; 429 = capacity issue	5xx > 2% for 5 min
Token usage (input/output per request)	Prompt bloat, loops	Avg input tokens +50% day-over-day
Cost (per request / feature / tenant)	Business health	Daily spend > budget × 1.3
Retrieval quality (hit rate, empty results, scores)	RAG health	"No relevant docs" rate doubles
Eval scores (online / sampled LLM-judge, feedback)	Output quality drift	Faithfulness drops below 0.85

COMMON GALTI

Sirf average latency dekhna. Average 2s ho sakta hai jab ke 5% users 20s wait kar rahe hon. Hamesha **p95/p99** bolo.

5.5 Alert, rollback ya investigate?

Situation	Decision
Recent deploy ke foran baad errors/quality gira, impact high	**Rollback** — pehle users bachao, phir dekho
SLO tezi se toot raha hai (error budget burn rate high)	**Alert / page** on-call
Dheere dheere drift, koi recent change nahi	**Investigate** — data/query shift, provider model change
Ek khaas tenant/feature affected	Feature flag off / scoped mitigation, phir investigate
Noise (ek-aadh spike, khud theek)	Alert nahi — dashboard par track; alert fatigue se bacho

INTERVIEW MEIN AISE BOLEIN

"Main alerts ko **symptoms** par lagata hoon jo user mehsoos karta hai — latency, errors, quality — har internal metric par nahi. Rollback ka rule simple hai: agar issue kisi recent change se correlate karta hai aur impact real hai, to pehle rollback; debugging rollback ke baad aaram se."

5.6 Proprietary API vs self-hosted — real traffic aur privacy ke saath

PRACTICE SCENARIO

Scenario: Bank, 30 lakh requests/month, average 2,000 input + 300 output tokens, customer data region se bahar nahi ja sakta.

Sochne ka tareeqa:

- **Privacy:** Public API out. Options: managed platform same region mein (private endpoint) ya self-hosted.
- **Volume:** 30 lakh × 2,300 ≈ 6.9 billion tokens/month — steady, high volume. Self-hosting economics yahan kaam kar sakti hai agar GPUs achhe utilize hon.
- **Quality:** Kya open-weight model eval set par required bar cross karta hai? Agar nahi, managed frontier model in-region.
- **Ops:** Kya team ke paas GPU serving aur on-call ki capacity hai?
- **Decision:** Dono ko apne eval set + load test par compare karo; cost per 1M tokens, p95 latency, aur quality ek table mein rakh kar defend karo.

YAAD RAKHNE WALI BAAT

Interviewer "sahi jawab" nahi dhund raha — wo dekh raha hai ke aap ne **privacy, volume, quality, ops capacity aur cost** sab ko explicitly weigh kiya ya nahi.

MODULE M15.6 · 2 SESSIONS

AI System-Design Interviews

Requirements se architecture, bottlenecks, failure modes aur scaling tak — ek structured AI system-design interview chalana.

06

Outcome: Structured AI-system-design interview chalana: requirements → architecture → bottlenecks → failure modes → scaling.

6.1 Ambiguity ko explicit assumptions mein badalna

Interviewer jaan boojh kar vague brief deta hai: *"Hamare employees ke liye ek AI assistant banao."* Pehle 5–7 minute **clarifying questions** mein lagao. 8 dimensions:

Dimension	Sawal
Users	Kaun? Kitne? Internal ya external? Kaun si languages?
Traffic	Daily active users? Peak requests per second? Spiky ya steady?
Latency	Chat (2–5s okay, streaming) ya real-time (<1s) ya batch (minutes okay)?
Quality	Ghalat jawab ki qeemat kya hai? Citations chahiye? Kitna accuracy acceptable?
Cost	Per-user ya monthly budget? Cost per query ka target?
Freshness	Data kitni jaldi update hota hai? Minutes, din, mahine?
Privacy	PII? Regulated data? Public API allowed? Data residency?
Reliability	Uptime target? Down ho to kya hoga? Fallback acceptable?

INTERVIEW MEIN AISE BOLEIN

"Shuru karne se pehle main kuch assumptions likh leta hoon — aap correct kar dijiye: 5,000 internal users, peak 20 requests/second, English + Urdu, policy documents ~50,000, jo hafte mein update hote hain. Ghalat jawab ka risk medium hai, is liye citations zaroori. Public API allowed hai lekin PII mask hogi. Target p95 latency 5 seconds streaming ke saath."

6.2 Sab se simple viable architecture pehle

Complexity ki seedhi — har step tab chadho jab pichla step **measurably** kaafi na ho:

1. Prompt + LLM → 2. + RAG → 3. + Tools → 4. Workflow → 5. Agent → 6. Fine-tune → 7. Routing, caching, multi-model

INTERVIEW MEIN AISE BOLEIN

"V1 mein main sirf RAG + ek frontier model rakhunga, aur pehle din se eval set aur tracing. Jab data dikhayega ke 30% queries simple FAQs hain, tab main router aur cache add karunga taake wo sasta model handle kare. Agent sirf tab jab users actions maangein, sirf jawab nahi."

COMMON GALTI

Pehle hi minute mein "multi-agent + fine-tuned Llama + Kafka + GraphRAG" bol dena. Interviewer isay over-engineering samajhta hai.

6.3 Full AI stack ko whiteboard karna

Ye 10 components aap ke whiteboard par hone chahiye — har ek ke saath **ek decision aur us ki wajah**:

Component	Kya decide karna hai
Model selection / routing	Kaun sa model kis query ke liye? Easy queries chhote model ko, hard bare ko. Fallback model.
RAG & context	Chunking, hybrid search, reranking, kitna context, citations, ACL filtering
Tools / agents	Kaun se actions? Workflow ya agent? Permissions?
Caches	Exact, semantic, prompt caching, embedding cache — freshness aur ACL ke saath
Databases	Vector DB, metadata DB, chat history, object store
Inference services	API vs managed vs self-hosted; streaming; timeouts
Observability	Traces per stage, latency, tokens, cost, user feedback
Evals	Offline golden set (CI gate), online sampled evals, A/B tests
Security	AuthN/Z, prompt injection defense, PII redaction, output filtering, audit logs
Deployment	Containers, autoscaling, canary, rollback, versioned prompts/indexes

6.4 Enterprise constraints ke saath design (public API allowed nahi)

Bohat se banks, hospitals, government projects mein public API band hoti hai. Do raaste — aur 4 dimensions par defend karna hai:

Dimension	Managed platform (Azure OpenAI, Bedrock, Vertex — in-tenant)	Self-hosted (open-weight on own GPUs)
Identity	Cloud IAM / Entra ID se native integration, managed identities	Khud SSO, RBAC, service accounts setup
Residency	Region choose karo; private endpoints; provider ki data-use policies	Data 100% aap ke data center/VPC mein

Capacity	Quotas (TPM) aur provisioned throughput; region mein availability	GPU khareedna/reserve karna; scaling aap ki zimmedari
Cost	Pay-per-token ya provisioned units; ops sasta	CapEx/GPU-hours + engineers; high utilization par sasta

INTERVIEW MEIN AISE BOLEIN

"Agar compliance team managed cloud allow karti hai, to main in-region managed platform choose karunga private endpoint ke saath — frontier quality milti hai aur ops kam. Agar policy kehti hai ke data hamare data center se bahar hi nahi jayega, to self-hosted open-weight model, aur main eval set par dikhaunga ke us ki quality kaafi hai."

6.5 Trade-offs explain karo, sirf tools ke naam nahi

Kamzor jawab

"Main Pinecone use karunga, LangChain, aur GPT."

Strong jawab

"Mujhe managed vector DB chahiye kyunke team chhoti hai aur ops kam rakhna hai; agar hum pehle se Postgres chala rahe hain to pgvector bhi theek hai — 50 lakh vectors tak achha chalega aur ek kam system maintain karna parega. Trade-off: bohat bare scale par dedicated DB behtar."

Formula: **"Main X choose karunga kyunke [requirement]. Is ka nuqsaan Y hai, jo is case mein acceptable hai kyunke [reason]. Agar [condition] badle, to main Z par jaunga."**

6.6 Practice ke liye 4 systems (mini designs)

A Enterprise support assistant

Requirements: Employees/customers ke sawal, company docs + ticket history se jawab, actions (ticket banana), escalation.

Architecture: Gateway (SSO) → orchestrator → intent router (FAQ / doc-question / action) → hybrid RAG with ACL filters + reranker → LLM with citations → tools (create_ticket, check_status) → human handoff.

Key trade-offs: semantic cache (sasta) vs freshness; chhota model for FAQs vs quality; agent vs workflow for actions (workflow choose).

Failure modes: stale docs, ghalat permissions par data leak, hallucinated policy, escalation miss.
Evals: faithfulness, answer correctness, resolution rate, escalation precision.

B Coding assistant

Requirements: IDE mein autocomplete (bohat low latency) + chat (codebase ke sawal) + agentic edits (tests chala kar fix).

Architecture: Autocomplete ke liye chhota, tez model (<300ms), FIM (fill-in-middle); chat ke liye bara model + **code-aware retrieval** (AST/symbol index, embeddings, grep); agent mode sandboxed container mein tests run kare.

Key trade-offs: latency vs quality (2 models), context selection (poori repo fit nahi hoti), privacy (code bahar jaye ya nahi), agent permissions (sandbox, no prod access).

Evals: acceptance rate, tests pass rate, edit success on benchmark tasks, latency p95.

C Research agent

Requirements: Ek sawal par web + internal sources se research karke cited report.

Architecture: Planner (sub-questions) → parallel search sub-agents (saaf context har ek ka) → reader/extractor → synthesizer with citations → verifier (claims vs sources). Budget aur max steps.

Key trade-offs: depth vs cost/time; multi-agent parallelism vs token cost; source quality filtering.

Failure modes: runaway search, low-quality sources, citation hallucination, prompt injection web pages se. **Evals:** citation accuracy, coverage, factuality, cost per report.

D Large-scale document intelligence system

Requirements: Lakhon invoices/contracts/forms se structured data nikalna, validate karna, downstream systems mein bhejna.

Architecture: Batch/async pipeline: object store → queue → classifier (doc type) → OCR/layout parser → extraction (LLM with JSON schema, ya fine-tuned chhota model) → validation rules → confidence threshold → human review queue for low-confidence → ERP/DB.

Key trade-offs: throughput aur cost (batch APIs, chhota FT model) vs accuracy; human review ka percentage vs automation rate.

Evals: field-level precision/recall, straight-through-processing rate, cost per document, processing time.

MODULE M15.7 · 1 SESSION

Staying Current Without Chasing Hype

Har hafte naya model, naya framework. Interviewer dekhna chahta hai ke aap ke paas naye tools ko **judge** karne ka process hai — sirf list nahi.

07

Outcome: Informed judgment demonstrate karna — sirf latest tools ki list sunane ke bajaye.

7.1 Nayi cheez ko evaluate karne ka defensible process

Naya model, framework, paper, benchmark ya tool — sab ke liye ek hi filter chalao:

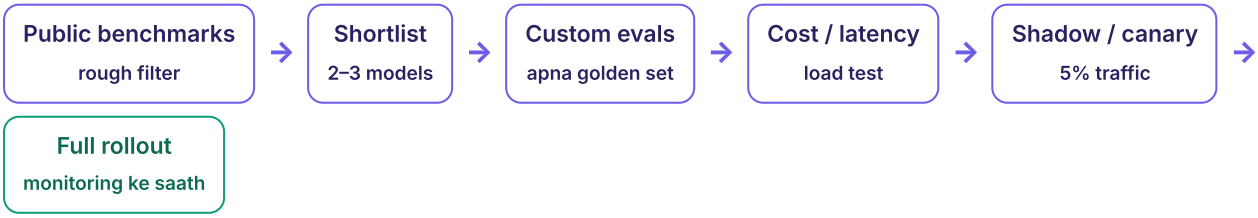
1. **Kya ye mera koi asal masla solve karta hai?** (Quality, cost, latency, reliability, developer speed.)
2. **Evidence kya hai?** Official docs/paper, reproducible results, independent evals — sirf tweets/demos nahi.
3. **Apne data par chhota test** — 1–2 din ka time-boxed spike.
4. **Adoption cost:** migration, lock-in, maturity, community, security review.
5. **Decision likho:** adopt / watch / ignore — aur kab dobara dekhna hai.

7.2 Capability change vs hype

Meaningful change (dhyaan do)	Short-lived hype (ruk kar dekho)
Same quality par cost 5–10× kam	Ek cherry-picked demo video
Context window / tool use / reasoning mein naya capability jo pehle mumkin nahi tha	"X is dead" type headlines
Naya standard jo ecosystem adopt kar raha hai (e.g., MCP)	Har hafte naya wrapper framework jo wahi kaam karta hai
Infra improvement (serving 3× tez, sasta quantization)	Sirf ek benchmark par 1–2% improvement

Test: "Kya 6 mahine baad bhi ye relevant hoga? Kya is se mere system ka koi number (cost, latency, quality) measurably badlega?"

7.3 Naya release hua model — production model replace karne se pehle



- **Benchmarks** sirf shortlist ke liye — wo aap ke data par nahi bane, aur kabhi contaminated bhi hote hain.
- **Custom evals** asal faisla karte hain: aap ki queries, aap ka format, aap ke edge cases.
- Prompts naye model ke liye **re-tune** karne par sakte hain — purane prompt par compare karna unfair ho sakta hai.
- **Shadow mode** (dono models chalao, user ko purana dikhao) ya canary — phir metrics compare.
- Rollback ready rakho; model gateway ke peeche switch config se ho.

7.4 "How do you stay current in GenAI?" — strong answer

INTERVIEW MEIN AISE BOLEIN

"Main do layers mein follow karta hoon. **Signal sources:** major labs ke official release notes aur docs, kuch selected papers jo practitioners discuss kar rahe hon, aur engineering blogs jahan log production experience share karte hain. **Filter:** har nayi cheez par main ek sawal poochta hoon — kya is se mere system ka cost, latency ya quality measurably badlega? Agar haan, to main apne eval set par ek time-boxed test karta hoon. Misaal ke taur par, jab [naya model] aaya, main ne usay apne 100-question golden set par test kiya — quality barabar thi lekin cost ~40% kam, is liye canary ke baad switch kiya. Social media mujhe batata hai ke *kya* aaya; mera eval set batata hai ke *kya kaam ka hai*."

COMMON GALTI

Sirf naam ginwana: "Main Twitter, YouTube, newsletters follow karta hoon." Process aur ek real example ke bagair ye jawab kamzor hai.

MODULE M15.8 · 2 SESSIONS

Project & Portfolio Defense

Projects ko "dekho ye chalta hai" ki tarah nahi, balkay **engineering decisions** ki tarah defend karna.

08

Outcome: Projects ko engineering decisions ki tarah defend karna — sirf ye dikhana kaafi nahi ke wo kaam karte hain.

8.1 Defend karne ke liye 2–3 projects chunna

Kam projects, lekin **deep**. Achha mix:

1 · RAG / knowledge system
Ingestion, retrieval quality, evals, citations.

2 · Agent / automation
Tools, guardrails, failure handling, HITL.

3 · Production-ish project
Deployed, real users ya real data, monitoring, cost numbers.

Har project ke baare mein aap ko **45 minute tak** sawal jhelne ke qabil hona chahiye. Agar kisi project ke andar aap ko khud har decision ki wajah nahi pata, wo project resume se hata do ya pehle us ko deeply samjho.

8.2 Har project explain karne ka 9-step structure

Problem → Requirements → Baseline → Architecture → Decisions → Trade-offs → Failures → Evals → Production results

WORKED EXAMPLE — WHATSAPP AI SUPPORT BOT (SHOPIFY STORE)

Ye template apne asal project par bharo; [brackets] mein apne asal numbers daalo.

- **Problem:** Online store par roz [X] WhatsApp messages — order status, delivery, COD, returns. Owner manually jawab deta tha, response time ghanton mein.
- **Requirements:** Roman Urdu + English, order data real-time, 24/7, kam cost, ghalat order info bilkul nahi.
- **Baseline:** Simple keyword auto-replies — [Y]% sawal handle hote the.
- **Architecture:** WhatsApp webhook → backend → intent classification → tools (order lookup via store API) + FAQ RAG (policies) → LLM reply → human handoff.
- **Decisions:** Order info hamesha **tool se**, LLM ki yaad se kabhi nahi; workflow over agent kyunke intents limited hain.
- **Trade-offs:** Chhota sasta model vs bara model; latency vs quality; Roman Urdu handling.
- **Failures:** e.g., phone number format mismatch se order nahi mila → normalization add ki.
- **Evals:** [N] real customer messages ka test set; intent accuracy, correct-answer rate.
- **Results:** [Z]% messages bina insaan ke resolve; response time ghanton se seconds; cost per conversation [Rs ...].

8.3 Har technology choice justify karna

Har choice ke liye 3 cheezein tayyar rakho: **kyun ye**, **kya alternative socha**, **kab doosra choose karte**.

Choice	Example justification
Model	"Eval set par GPT-class aur chhote model ka farq 2% tha lekin cost 8× kam — is liye chhota."
Retriever	"Hybrid (BM25 + vector) kyunke users order IDs aur SKU codes likhte hain jo pure embeddings miss karti hain."
Agent architecture	"Workflow, kyunke 5 fixed intents the — agent ki unpredictability ki zaroorat nahi."
Framework	"Framework kam rakha / plain SDK, kyunke debugging aur control asaan."
Database	"Postgres + pgvector — ek hi DB, data chhota hai, ops kam."
Deployment	"Serverless/container on [platform] — traffic kam aur spiky, idle cost zero."

8.4 Outcome quantify karna

Numbers ke bagair project "toy" lagta hai. Jo bhi measure kar sakte ho:

Quality score / accuracy · Faithfulness · Latency p95 · Cost per query · Throughput · Dataset size · Traffic / users · Improvement over baseline · Time saved

AGAR REAL USERS NAHI HAIN

Koi baat nahi — apna **eval set** banao (50–200 examples), baseline vs final compare karo, aur load test se latency/throughput nikalo. "Baseline 54% se 81% correct answers, p95 latency 3.2s, cost Rs 0.9 per query" — ye simulated numbers bhi bohat strong lagte hain jab honestly batao ke kaise measure kiye.

8.5 Failures openly discuss karna

Interviewer jaan boojh kar poochta hai: "Is project mein kya ghalat hua?" "Kuch nahi" sab se kharab jawab hai. Format:

INTERVIEW MEIN AISE BOLEIN

"Pehle version mein main ne fixed 1,000-token chunks rakhe the. Eval mein dekha ke table wale sawal 40% fail ho rahe the kyunke tables beech se kat rahi thi. Main ne structure-aware chunking ki aur table headers har chunk mein repeat kiye — table questions ka score 58% se 84% gaya. Seekha ye ke chunking strategy data ki shape se aani chahiye, default value se nahi."

8.6 Expected follow-up sawal — aur jawab ka framework

Q "Why didn't you use RAG?"

Batao ke masla knowledge ka tha ya nahi. "Mera task classification/extraction tha — model ko external knowledge nahi chahiye thi, sirf input document. RAG latency aur complexity add karta bina faide ke." Ya agar RAG lagana chahiye tha: "Fair point — data badhne ke baad ye zaroori ho jayega; main X point par add karunga."

Q "Why not fine-tune?"

"Prompt + few-shot se [X]% mil gaya jo requirement se upar tha. FT ke liye labeled data, training aur serving infra chahiye tha — us ka ROI nahi tha. Agar volume 10× ho jaye ya format consistency ka masla aaye, to chhota model fine-tune karunga."

Q "Why do you need an agent?"

Agar agent use kiya: batao ke path pehle se pata nahi tha (number of steps vary karta tha). Agar nahi kiya: "Maine jaan boojh kar workflow rakha — reliability aur cost ke liye."

Q "What happens at 100× traffic?"

Bottlenecks order mein batao: (1) LLM rate limits/cost pehle toote ga → caching, routing, provisioned throughput, multiple providers; (2) vector DB QPS → replicas/sharding; (3) ingestion → queue workers scale; (4) DB connections → pooling. Stateless services horizontally scale.

Q "How would you reduce the cost by 50%?"

Pehle measure: cost kahan ja raha hai (input tokens? output? kaun sa feature?). Phir levers: prompt chhota, retrieved chunks kam/behtar reranking, semantic cache, prompt caching, easy queries chhote model ko (routing), batch APIs for offline work, output length limit, chhota fine-tuned model. Har lever ke saath quality par eval check.

Q "What breaks first?"

Apne system ka sab se kamzor point honestly batao: "Sab se pehle third-party API rate limit — kyunke mere paas abhi ek hi provider hai. Phir ingestion, kyunke wo synchronous hai. Main inko pehle fix karunga: fallback provider aur async queue."

MODULE M15.9 · 1 SESSION

Behavioral Communication for AI Engineers

Strong technical thinking tab hi kaam ki hai jab interviewer usay **dekh** sake.

09

Outcome: Apni strong technical thinking ko interviewer ke liye visible banana.

9.1 Clarity — implementation details mein dubaye bina

Pehle **headline**, phir detail agar poocha jaye. "Top-down" bolo: "Mera design teen hisson mein hai: ingestion, retrieval, generation. Pehle ingestion..." Function names, library versions, code lines — jab tak poocha na jaye, mat bolo.

60-SECOND RULE

Har answer ka pehla version ~60 second ka ho. Phir poocho: "Kya main kisi hisse mein deep jaun?" Is se interviewer conversation steer karta hai aur aap ka waqt sahi jagah lagta hai.

9.2 Pehle assumptions batao

Ambiguous sawal par seedha solution nahi. "Main assume kar raha hoon ke ... — kya ye theek hai?" Is se do faide: ghalat direction mein 20 minute zaya nahi hote, aur interviewer ko structured thinking dikhti hai.

9.3 Trade-offs mein socho

Tool ya architecture par seedha jump nahi. Pehle options, phir criteria, phir choice:



9.4 Diagram ko progressively reveal karna

Whiteboard par pehle **4–5 boxes** ka simple flow (user → API → retrieval → LLM → response). Phir har round mein ek layer add karo: cache, queue, observability, fallbacks. Poora complex diagram ek dum se banane se interviewer kho jata hai — aur aap bhi.

9.5 Bluff nahi — measure karne ka tareeqa batao

Kamzor

"Haan, chunk size 512 best hota hai." (bina wajah confident)

Strong

"Mujhe pakka nahi pata is data ke liye kaun sa best hai. Main 256, 512 aur structure-based chunking ko apne eval set par recall@5 se compare karunga — data khud bata dega."

"Mujhe nahi pata, lekin main aise pata karunga" — ye senior engineers ka jawab hai. Bluff pakra jaye to poora interview ka trust jata hai.

9.6 Experience-based answers ka structure

"Ek waqt batao jab..." type sawalon ke liye 5 hisse:

Hissa	Kya bolna hai	Waqt
Situation	Context: project, masla, kyun important tha	~15%
Decision	Kya options the, aap ne kya choose kiya aur kyun	~25%
Implementation	Aap ne **khud** kya kiya ("main", "hum" nahi)	~25%
Result	Numbers ke saath outcome	~20%
Lessons	Kya seekha, agli baar kya alag karte	~15%

COMMON BEHAVIORAL SAWAL — TAYYAR RAKHO

- Ek technical decision jo baad mein ghalat nikla
- Jab aap ka system production mein toota
- Jab aap ko kisi stakeholder ko "nahi" bolna para (e.g., "agent nahi chahiye")
- Jab deadline aur quality mein trade-off karna para
- Jab aap ne koi bilkul nayi technology jaldi seekhi

MODULE M15.10 · 2 SESSIONS · SHIP IT

Capstone: Full Mock GenAI Interview Loop

Ek complete AI-engineering interview simulate karo — 3 rounds. Maqsad: dikhana ke aap AI engineer ki tarah **sochte** ho, sirf concepts yaad nahi.

10

Outcome: Interview conditions mein demonstrate karna ke aap AI engineer ki tarah reason kar sakte ho — sirf AI concepts recall nahi.

Round 1 · Scenario & debugging

6 rapid production scenarios, har ek 4–5 minute.

Round 2 · System design

Ambiguous brief → 10-step design, ~40 minute.

Round 3 · Project defense

Ek project, gehre se gehre follow-ups, ~30 minute.

MOCK KAISE KAREIN

Kisi dost/colleague ko interviewer banao (ya AI ko interviewer role do), timer lagao, video/audio record karo. Har round ke baad neeche wale rubric se khud ko score karo.

Round 1 — Rapid production scenarios (model answers)

1 RAG accuracy suddenly drops.

Mitigate: Agar kal deploy hua tha (prompt/model/index), to rollback. **Clarify:** sab queries ya kuch topics? Kab se? **What changed:** deploys, provider model version, embedding model, data ingestion (naye docs, failed parses, purane versions delete na hona). **Isolate:** failing queries par check — sahi chunk top-k mein hai? Nahi → retrieval/ingestion. Haan → generation/prompt/model. **Guard:** failing cases eval set mein, model version pin, ingestion health alerts.

2 Agent enters an expensive loop.

Mitigate: Run kill karo, global per-run budget aur max-steps limit foran lagao. **Diagnose:** trajectory dekho — kya same tool same arguments ke saath bar bar? Aam wajah: tool error message vague hai, agent ko samajh nahi aata ke kaam "done" hai, ya success condition clear nahi. **Fix:** clear termination criteria, structured tool errors ("ye order exist nahi karta — dobara mat try karo"), repeated-action detection, stuck hone par human escalation. **Guard:** cost-per-run alert, loop scenario eval mein.

3 Model provider latency doubles.

Confirm: traces se dekho ke latency waqai LLM span mein hai (retrieval mein nahi), provider status page. Kya hamare prompts lambe ho gaye (input tokens barhe)? **Mitigate:** streaming ensure, timeouts, traffic ka hissa secondary provider/region par, easy queries chhote/tez model ko route, cache hit rate barhao. **Long-term:** multi-provider gateway, provisioned throughput, latency SLO alerts.

4 Fine-tuned model regresses after deployment.

First test: wahi eval set production serving stack par chalao. Score gira → serving issue: chat template mismatch, tokenizer, generation params, adapter version, quantization. Score theek → data issue: distribution shift, eval set production ko represent nahi karta, leakage. **Mitigate:** pichla model/version rollback. **Guard:** deployment pipeline mein "serving-stack eval" gate.

5 Prompt change improves quality but doubles cost.

Business sawal pehle: quality improvement kitna valuable hai? (e.g., escalations 20% kam = support cost bachat.) **Options:** (a) nayi prompt sirf hard queries par (routing), (b) prompt compress karo — kaun sa hissa asal mein improvement de raha hai? Ablation karo, (c) prompt caching agar static prefix bara hai, (d) chhota model naye prompt ke saath test. **Decision:** quality per rupee compare karke A/B test, numbers ke saath stakeholder ko trade-off dikhao.

6 Retriever works offline but fails on real production queries.

Wajah: offline eval set production queries jaisa nahi — real users chhote, vague, typos, Roman Urdu, IDs/codes, multi-part sawal poochte hain. **Diagnose:** production logs se failing queries sample karke categorize. **Fix:** query rewriting/expansion, hybrid search (BM25 exact codes ke liye), reranker, metadata filters, multilingual embeddings. **Guard:** eval set ko real production queries se refresh karo (har mahine).

Round 2 — System design walkthrough

AMBIGUOUS BRIEF

"Ek bank chahta hai ke uske call-center agents ke liye AI assistant ho jo policies aur customer ke account context se jawab dhoondne mein madad kare."

#	Step	Is brief par kya bolna hai
1	Clarify requirements	Users = 800 call-center agents (customers nahi). Use: policy lookup + account summary. Languages: Urdu/English. Real-time call ke dauran — latency matter karti hai.

2	Define constraints	Customer PII — public API nahi; data in-country/region. p95 < 3s. Ghalat policy batana high risk → citations zaroori. Peak ~15 RPS.
3	Establish baseline	Abhi agents PDF/intranet search karte hain ~2 min per query. Target: 15 seconds mein cited answer. Golden set: 200 real agent questions with correct answers.
4	Design architecture	Agent desktop → gateway (SSO) → orchestrator → [policy RAG] + [account tool: read-only core-banking API] → LLM → cited answer. Async ingestion for policies.
5	Choose models	In-region managed platform ya self-hosted open-weight (compliance decide karega); eval set par compare. Embeddings multilingual.
6	Data / context layer	Policies: structure-aware chunks + version metadata (sirf current version retrieve), hybrid search + reranker. Account data: tool se live, LLM context mein sirf zaroori fields, PII masked logs.
7	Handle scale	Stateless services autoscale; semantic cache for common policy questions (account-specific nahi); provisioned throughput; read replicas.
8	Reliability & security	Fallback: LLM down → sirf retrieved policy snippets dikhao. Read-only tool permissions, prompt injection checks, audit log, RBAC, encryption, secrets manager.
9	Evals & monitoring	Offline: faithfulness, correctness, citation accuracy on golden set (CI gate). Online: agent thumbs up/down, latency p95, cost/query, "no answer" rate.
10	Defend trade-offs	Agent nahi, workflow (predictable, auditable). Fine-tune nahi (knowledge problem → RAG). Cache sirf non-personal questions par (privacy). Self-hosted vs managed: compliance + eval data se decide.

Round 3 — Project defense drill

Interviewer har jawab ke baad aur gehra jata hai. Apne project par in 4 levels ki practice karo:

Depth	Example follow-ups
Level 1 · What	Project kya karta hai? Architecture draw karo.
Level 2 · Why	Ye model kyun? Ye DB kyun? Agent kyun / kyun nahi? Alternatives kya the?
Level 3 · How well	Kaise pata ke kaam karta hai? Eval set kitna bara? Baseline kya tha? Cost per query?
Level 4 · What if	100× traffic? Cost aadhi karni ho? Provider band ho jaye? Naye language users aayein? Sab se pehle kya tootega?

Self-scoring rubric (har mock ke baad)

Skill	1 — Weak	3 — OK	5 — Strong	Score
Clarifying	Seedha solution	Kuch sawal	Assumptions likhe, confirm kiye	
Structure	Random order	Kuch flow	Clear framework, top-down	
Simplicity	Over-engineered	Theek	Simplest first, phir justified complexity	

Trade-offs	Sirf tool names	Kuch pros/cons	Har decision ke saath trade-off + condition
Debugging	Guesses	Partial isolation	Mitigate → what changed → isolate → guard
Measurement	Koi metric nahi	Generic metrics	Specific evals, SLOs, numbers
Communication	Jargon, lamba	Theek	Clear, concise, diagram progressively
Honesty	Bluff	Kabhi kabhi	"Pata nahi, aise measure karunga"

REVISION

Rapid-Fire Revision: 20 Sawal, Chhote Jawab

Interview se ek din pehle ye page parho. Har jawab 1–2 line mein bolne ki practice karo.

Sawal	Short answer
RAG vs fine-tuning ek line mein?	RAG = knowledge/facts; FT = behavior/format/style. Pehle prompt.
Hybrid search kyun?	Embeddings meaning pakarti hain, BM25 exact words/IDs/codes — dono mila kar recall behtar.
Reranker kya karta hai?	Top-50 candidates ko cross-encoder se dobara score karke best top-5 deta hai; precision barhta hai, latency thori barhti hai.
Stateless service kyun?	Koi bhi replica koi bhi request serve kare → horizontal scaling, asaan restart.
Semantic cache ka risk?	Milte julte lekin alag sawal ka ghalat jawab; ACL/personal data leak; stale answers.
Retry kab nahi karna?	4xx validation errors par; non-idempotent writes par bina idempotency key.
Circuit breaker?	Lagataar fail hone wali dependency ko temporarily call karna band → fallback; retry storm se bachao.
p95 latency kyun, average kyun nahi?	Average tail ko chhupa leta hai; 5% users ka bura experience p95/p99 dikhata hai.
TTFT?	Time to first token — streaming UX ka sab se important latency number.
KV cache?	Pichle tokens ke attention keys/values store — generation tez, lekin GPU memory context × users ke saath barhti hai.
LoRA ka serving faida?	Ek base model par kai chhote adapters — multi-tenant, sasti serving.
QLoRA kab?	GPU memory kam ho aur bara model train karna ho (4-bit base + LoRA).
Quantization ka risk?	Edge cases/reasoning par accuracy drop — deploy se pehle quantized version ka eval.
Workflow vs agent?	Steps pehle se pata → workflow. Path open-ended → agent. Default workflow.

MCP kya hai?	Tools/resources expose karne ka open standard — ek baar banao, kai clients use karein.
Agent ke liye 3 safety controls?	Least-privilege tools, step/budget limits, high-risk actions par human approval.
Prompt injection defense?	Untrusted content ko instructions se alag rakho, tool permissions limit, output/action validation, high-risk par HITL.
Offline vs online evals?	Offline = golden set par CI gate; online = production traffic sample, feedback, drift detection.
Naya model kaise adopt karein?	Benchmarks → shortlist → custom evals → cost/latency → canary → rollout.
Cost 50% kam kaise?	Measure pehle; phir routing, caching, prompt/context trim, prompt caching, batch API, chhota FT model.

Interview se pehle final checklist

- ☐ 2–3 projects ka 9-step write-up yaad hai
- ☐ Har project ke numbers (quality, latency, cost) tayyar
- ☐ Har project ki ek failure story + lesson
- ☐ 6 follow-up sawalon ke jawab practice kiye
- ☐ RAG architecture 5 minute mein whiteboard kar sakta hoon
- ☐ Debugging framework: mitigate → changed → isolate → guard
- ☐ API vs self-hosted ke 6 dimensions
- ☐ Agent ke 7 failure modes + fixes
- ☐ "How do you stay current?" ka jawab example ke saath
- ☐ Kam az kam 3 full mock loops record karke review kiye

AAKHRI BAAT

Interviewer perfect jawab nahi dhundta. Wo ye dekhta hai ke kya aap ke saath kaam karna aasaan hoga jab production mein raat 2 baje kuch toote: kya aap **calm** rehte ho, **structured** sochte ho, **measure** karte ho, aur **honest** ho. Ye PDF isi cheez ki practice hai. All the best!